Java — Séance 1

Les bases du langage

Du premier `void main()` au premier test JUnit

Le module : 4 séances de 4 heures

Séance	Thème
1 — aujourd'hui	Les bases du langage
2 — dans un mois	Du code aux objets
3	Robuste et persistant
4	Plusieurs clients, SQL et sécurité 🔒

Chaque séance : **cours court** entrecoupé d'**ateliers** 🛠️ sur votre machine, puis **TP en classe**.
Le TP, fil rouge des 4 séances : **ShortLink**, un raccourcisseur de liens guidé pas à pas.

Comment vous êtes évalués

Plusieurs notes courtes, pas d'examen final. On note surtout ce que vous savez **expliquer**.

Quand	Quoi
Fin de séance 1	QCM **diagnostique**, non noté
Avant la séance 2	TP rendu sur GitHub : fait, et `mvn test` passe
Séances 2, 3, 4	QCM de 10 min sur la séance précédente
Séance 2	**3 min** avec moi : expliquer et modifier votre code du TP
Fin des séances 2 et 4	TP rendu sur GitHub, vérifié par des tests de correction
Séance 4	**3 min** avec moi : expliquer une correction de sécurité

Les règles du jeu

Vous êtes adultes, je vous traite en adultes.

- Vous gérez votre rythme : 🟢 pour tous, 🔵 🟣 pour aller plus loin.
- Vous êtes autonomes : vous cherchez, puis vous demandez **quand vous bloquez**.

L'IA est autorisée. Mais :

- **Vous êtes évalués sans elle** : QCM, et 3 minutes pour expliquer et modifier **votre** code.
- **Cherchez 15 min seul** avant de demander. Demandez une **explication**, pas du code.
- **Les tests (et Valkey) ont le dernier mot**, pas l'IA. Elle oublie souvent les cas limites.
- **Pas de code que vous ne savez pas expliquer** dans votre dépôt.

Objectifs de la séance

AA1 — Écrire, compiler et exécuter un programme Java, puis le construire et le tester avec Maven et JUnit.

0. 🛠️ Préparer son poste : Java 25, IntelliJ, débogueur
1. Du code source à la JVM : votre premier programme
2. Variables, types et opérateurs
3. Conditions, boucles, tableaux et méthodes
4. Chaînes de caractères et validation des entrées 🔒
5. Maven et un premier test JUnit

0. 🛠️ Préparer son poste

🛠️ Installer Java 25 et Maven avec SDKMAN

```
$ curl -s "https://get.sdkman.io" | bash
$ sdk list java                  → repérer Temurin 25, colonne Identifier
$ sdk install java 25.0.5.0+8-tem
$ sdk install maven
$ java -version                  → openjdk version "25…"
$ mvn -v                         → … Java version: 25…
```

- macOS, Linux et **WSL**. Sans droits administrateur.
- Windows sans WSL : IntelliJ télécharge le JDK (*Download JDK…*).
- `sdk use java …` change de version dans ce terminal seulement.

📖 Support §0.1 et §0.2 — Installer Java et Maven

🛠️ IntelliJ : un premier projet en 2 minutes

New Project	Valeur
Build system	**Maven**
JDK	**25** (sinon *Add JDK from disk* ou *Download JDK*)
Add sample code	coché
GroupId	`fr.foodfast`

- Code dans `src/main/java`, tests dans `src/test/java`.
- ▶️ dans la marge à côté de `main` : *Run*. Sortie en bas.
- Taper `main` + Tab, `sout` + Tab.

📖 Support §0.3 — IntelliJ IDEA

🛠️ Le débogueur : voir le programme s'exécuter

```java
public class BuggyTotal {
    static int totalCents(int[] pricesCents) {
        int total = 0;                          // ● point d'arrêt
        for (int i = 1; i < pricesCents.length; i++) {
            total = total + pricesCents[i];
        }
        return total;
    }

    public static void main(String[] args) {
        int[] basket = {1250, 450, 800};
        System.out.println(totalCents(basket)); // 2500 attendu… 1250 ?!
    }
}
```

1. Clic dans la marge : **point d'arrêt**. 2. ▶️ à côté de `main` → **Debug**.

2. **F8** : ligne suivante. 4. Regarder `i` et `total`. 5. **F9** (Mac : ⌥⌘R) : reprendre.

📖 Support §0.4 — Le débogueur

1. Du code source à l'exécution

Le chemin d'un programme Java

```
Hello.java  ──javac──→  Hello.class  ──java──→   JVM
(source,                (bytecode,              (exécute sur
 lisible)                portable)               Windows, Linux, macOS)
```

- `javac` : le compilateur. Il vérifie le code et produit du **bytecode**.
- **Bytecode** : un langage intermédiaire, le même sur toutes les machines.
- **JVM** (*Java Virtual Machine*) : exécute le bytecode. Elle l'interprète, puis compile à la volée (**JIT**) les parties souvent utilisées.
- La JVM gère la mémoire : le **ramasse-miettes** (*GC*) libère les objets inutilisés.
- On installe le **JDK 25** (LTS) : `javac`, `java`…

📖 Support §1.1 — JDK, bytecode et JVM

Votre premier programme (Java 25)

```java
// Fichier Hello.java
void main() {
    IO.println("Bienvenue chez FoodFast !");
}
```

```
$ javac Hello.java     → produit Hello.class
$ java Hello           → Bienvenue chez FoodFast !
$ java Hello.java      → raccourci : compile en mémoire et exécute
```

- **Compact source file** (JEP 512, final en Java 25) : pas de classe à écrire.
- `IO.println` affiche une ligne. `IO` est une classe de `java.lang` : aucun import.

📖 Support §1.2 — Le compact source file

La forme classique : une classe et `main`

```
public class FoodFastApp {
    public static void main(String[] args) {
        System.out.println("Bienvenue chez FoodFast !");
    }
}
```

Mot	Sens
`public class FoodFastApp`	une classe nommée, dans `FoodFastApp.java`
`static`	méthode de la classe, appelable sans objet
`void`	ne renvoie rien
`String[] args`	arguments de la ligne de commande

📖 Support §1.3 — La forme classique

Pourquoi une classe nommée dans un projet ?

Compact source file

- classe **implicite**, sans nom
- aucun autre code ne peut l'appeler
- donc **impossible à tester** avec JUnit

Classe nommée

- `OrderCalculator.totalCents(...)`
- appelable depuis une autre classe
- **testable**, rangée dans un projet Maven

> Le *compact source file* sert à apprendre et à écrire des scripts. Dès qu'on veut tester, on écrit une **classe nommée**.

🛠️ Lire une erreur : où, puis quoi

```
Hello.java:2: error: cannot find symbol
    IO.printn("Bienvenue chez FoodFast !");
      ^
  symbol:   method printn(String)
  location: class IO
```

- **Où** : fichier `Hello.java`, ligne `2`, colonne du `^`.
- **Quoi** : `cannot find symbol`, un nom inconnu : `printn`.
- Corriger la **première** erreur, puis recompiler.

🛠️ À vous — Partie 1 (15 à 20 min)

1. Dans un terminal

```
// Fichier Hello.java
void main() {
    IO.println("Bienvenue chez FoodFast !");
}
```

```
$ javac Hello.java
$ java Hello
```

Enlevez un `;`, recompilez, **lisez** l'erreur.

2. Dans IntelliJ

```
// Fichier FoodFastApp.java
public class FoodFastApp {
  public static void main(String[] args) {
    System.out.println("Bienvenue !");
  }
}
```

▶️ à côté de `main`. Changez le message, relancez.

🔵 Après `javac`, lancez `javap -c Hello` : le bytecode.

2. Variables, types et opérateurs

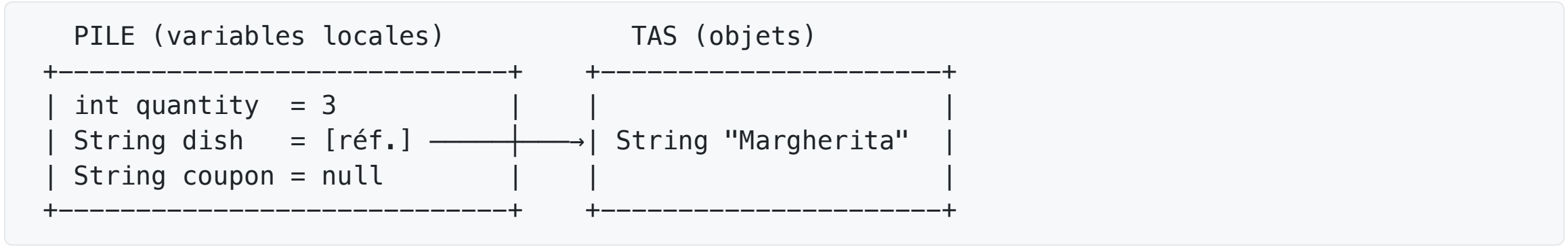
Les types primitifs utiles

Type	Contient	Exemple FoodFast
`int`	entier sur 32 bits (±2,1 milliards)	`int quantity = 3;`
`long`	entier sur 64 bits	`long orderCount = 12_000_000_000L;`
`double`	nombre à virgule (approché)	`double rating = 4.5;`
`boolean`	`true` ou `false`	`boolean delivered = false;`
`char`	un caractère Unicode	`char size = 'M';`

💶 Montants **en centimes** dans un `int` : `int priceCents = 1250;` (12,50 €). Pas de `double` pour l'argent ; `BigDecimal` arrive en séance 2.

Types références : `String` et `null`

```
int quantity = 3;              // primitif : la variable contient 3
String dish = "Margherita";    // référence : la variable désigne
                               // un objet String en mémoire
String coupon = null;          // null : ne désigne aucun objet
IO.println(dish.length());     // 10 : un objet a des méthodes
IO.println(coupon.length());   // ❌ NullPointerException
```



- Primitif : minuscule (`int`), pas de méthode, jamais `null`.
- Référence : majuscule (`String`), des méthodes, peut valoir `null`.

📖 Support §2.2 — Types références et null

`var` : laisser le compilateur déduire le type

```
var quantity = 3;              // int
var dish = "Margherita";       // String
quantity = "trois";            // ❌ ne compile pas : quantity est un int
var coupon;                    // ❌ ne compile pas : pas de valeur
var nothing = null;            // ❌ ne compile pas : type inconnu
```

- `var` **n'est pas** un type dynamique : le type est fixé à la compilation.
- Réservé aux **variables locales** initialisées dans la même instruction.
- À utiliser quand le type est évident à la lecture.

📖 Support §2.3 — var

Les opérateurs du quotidien

Catégorie	Opérateurs	Exemple
Arithmétique	`+ - * / %`	`total = price * quantity;`
Affectation	`= += -= ++ --`	`total += fee;` `count++;`
Comparaison	`== != < <= > >=`	`quantity <= 20`
Logique	`&& || !`	`qty >= 1 && qty <= 20`
Conversion	`(type)`	`(double) cents / 100`

> `&&` et `||` sont **paresseux** : `s != null && s.isBlank()` ne lève jamais de `NullPointerException`.

📖 Support §2.4 — Opérateurs

🤔 Prédisez la sortie

```
IO.println(7 / 2);
IO.println(7 % 2);
IO.println((double) 7 / 2);
IO.println("Total : " + 1 + 2);
IO.println(1 + 2 + " €");
```

🤔 Prédisez la sortie : `==` sur des `String`

```
String code = "PROMO10";
String typed = "promo10".toUpperCase();

IO.println(typed);
IO.println(typed == code);
IO.println(typed.equals(code));
```

`==` contre `equals`

❌ Compare les **références**

```
if (typed == code) {
    applyDiscount();
}
```

✅ Compare le **contenu**

```
if (code.equals(typed)) {
    applyDiscount();
}
```

- `==` pour les primitifs (`int`, `char`, `boolean`…).
- `equals` pour les objets (`String`…). `equalsIgnoreCase` ignore la casse.
- Constante à gauche : `"PROMO10".equals(typed)` ne plante pas si `typed` vaut `null`.

📖 Support §2.5 — == contre equals

🔓 Prédisez la sortie : le débordement d'entier

```
int priceCents = 1250;           // une pizza à 12,50 €
int quantity = 2_000_000;        // saisie malveillante
int total = priceCents * quantity;
IO.println(total);
```

- Un `int` va de –2 147 483 648 à **2 147 483 647**.
- Au-delà, Java ne signale **rien** : le calcul « fait le tour ».

🔒 Débordement : détecter au lieu de subir

❌ Débordement silencieux

```
int total = price * qty;
total = total + fee;
```

✅ Exception si ça déborde

```
int total = Math.multiplyExact(
        price, qty);
total = Math.addExact(total, fee);
```

- `Math.addExact`, `subtractExact`, `multiplyExact` lèvent une `ArithmeticException`.
- Le programme s'arrête au lieu de continuer avec un montant faux.
- Mieux encore : **valider la quantité** avant le calcul (§4.3).

🛠️ À vous — Partie 2 (20 à 30 min)

1. Prédire **sur papier**, puis lancer :

```
// Fichier Predict.java
void main() {
  IO.println(7 / 2);
  IO.println(7 / 2.0);
  IO.println("Total : " + 1 + 2);
  IO.println(Integer.MAX_VALUE + 1);
  IO.println(0.1 + 0.2);
}
```

2. `static String formatEuros(int cents)`

`2500` → `"25,00 €"` et `2505` → `"25,05 €"`

Indice : `cents / 100`, `cents % 100`.

3. Au débogueur : point d'arrêt, puis F9.

À quel tour `orders` devient négatif ?

```
void main() {
  int orders = Integer.MAX_VALUE - 2;
  for (int r = 1; r <= 5; r++) {
    orders = orders + 1;    // ●
    IO.println(orders);
  }
}
```

Puis `Math.addExact(orders, 1)`.

3. Conditions, boucles, tableaux, méthodes

`if` / `else` : frais de livraison

```
int subtotalCents = 2350;
int feeCents;

if (subtotalCents >= 3000) {
    feeCents = 0;                  // livraison offerte
} else if (subtotalCents >= 1500) {
    feeCents = 199;
} else {
    feeCents = 399;
}
IO.println("Frais : " + feeCents + " centimes");
```

- Accolades **toujours**, même pour une seule ligne.

📖 Support §3.1 — if / else

`switch` en expression

```
String zone = "PERIPHERIE";

int feeCents = switch (zone) {
    case "CENTRE" -> 199;
    case "PERIPHERIE", "BANLIEUE" -> 399;
    default -> 599;
};
IO.println(feeCents);          // 399
```

- `switch` **renvoie une valeur** ; flèche `->` : pas de `break` à oublier.
- Plusieurs valeurs par `case` ; `default` obligatoire pour une `String`.

📖 Support §3.2 — switch en expression

Boucles `for` et `while`

```java
// for : on connaît le nombre de tours
for (int table = 1; table <= 3; table++) {
    IO.println("Table " + table + " servie");
}

// while : on répète tant qu'une condition est vraie
int budgetCents = 3000;
int pizzas = 0;
while (budgetCents >= 1250) {
    budgetCents -= 1250;
    pizzas++;
}
IO.println(pizzas + " pizzas, reste " + budgetCents);  // 2, reste 500
```

📖 Support §3.3 — Boucles

Tableaux : le total d'une commande

```
int[] pricesCents = {1250, 990, 350};   // pizza, salade, boisson
int[] quantities  = {2, 1, 3};

int totalCents = 0;
for (int i = 0; i < pricesCents.length; i++) {
    totalCents += pricesCents[i] * quantities[i];
}
IO.println(totalCents);                  // 4540

for (int price : pricesCents) {          // for-each : lecture seule
    IO.println(price);
}
```

- Taille **fixe**, indices de `0` à `length - 1`.

📖 Support §3.4 — Tableaux

Méthodes `static` : nommer un calcul

```
static int totalCents(int[] prices, int[] quantities) {
    int total = 0;
    for (int i = 0; i < prices.length; i++) {
        total += prices[i] * quantities[i];
    }
    return total;
}

void main() {
    int t = totalCents(new int[] {1250, 350}, new int[] {2, 1});
    IO.println(t);                                          // 2850
}
```

- `static` + type de retour + nom + paramètres. `return` renvoie le résultat.

📖 Support §3.5 — Méthodes static

Composer des méthodes : appliquer une remise

```
static int applyDiscount(int totalCents, int percent) {
    if (percent < 0 || percent > 100) {
        throw new IllegalArgumentException(
                "Remise invalide : " + percent);
    }
    int discount = Math.multiplyExact(totalCents, percent) / 100;
    return totalCents - discount;
}
// applyDiscount(2850, 10)  → 2565
// applyDiscount(1999, 15)  → 1700   (299,85 tronqué à 299)
// applyDiscount(2850, 150) → IllegalArgumentException
```

- `throw` arrête la méthode avec une erreur explicite (détails en séance 3).

📖 Support §3.5 — Méthodes static

Aperçu : lire une `List` et une `Map`

```
List<String> words = List.of("ADD", "Margherita", "2");
words.size();                          // 3
words.getFirst();                      // "ADD"
words.get(1);                          // "Margherita"

Map<String, Integer> stock = new HashMap<>();
stock.put("Margherita", 12);           // ajoute ou remplace
stock.get("Margherita");               // 12
stock.get("Sushi");                    // null : clé absente !
stock.getOrDefault("Sushi", 0);        // 0
stock.containsKey("Sushi");            // false
stock.remove("Margherita");            // 12
```

🛠️ À vous — Partie 3 (30 à 40 min)

1. `static String dayLabel(int day)` avec un `switch` :
   `1` → `"lundi"` … `7` → `"dimanche"`, sinon `"jour invalide"`.
2. `static long sumUpTo(int n)` : `1 + 2 + … + n`. `sumUpTo(100)` vaut `5050`.
   Suivez `sumUpTo(4)` **au débogueur**. Pourquoi `long` et pas `int` ?
3. 🔵 Une classe `BasketStats`, avec `{1250, 450, 800, 1590}` :
   - `maxCents(prices)` → `1590`
   - `countAbove(prices, 1000)` → `2` (strictement au-dessus)
   - `averageCents(prices)` → `1022`
   - et sur un tableau **vide** ? À vous de décider.



Pause — 15 minutes

Reprise : **partie 4, chaînes de caractères et validation**

4. Chaînes de caractères et validation

Les méthodes de `String` à connaître

Méthode	Exemple	Résultat
`length()`	`"Pizza".length()`	`5`
`charAt(i)`	`"Pizza".charAt(0)`	`'P'`
`substring(a, b)`	`"order-123".substring(6)`	`"123"`
`indexOf(s)`	`"order-123".indexOf("-")`	`5`
`toUpperCase()`	`"promo10".toUpperCase()`	`"PROMO10"`
`strip()` / `isBlank()`	`"  3 ".strip()`	`"3"`
`startsWith(s)`	`"order-123".startsWith("order-")`	`true`

> Une `String` est **immuable** : ces méthodes renvoient une **nouvelle** chaîne.

📖 Support §4.1 — Méthodes de String

`StringBuilder` : construire un ticket

```
static String formatEuros(int cents) {
    return String.format("%d,%02d €", cents / 100, cents % 100);
}

static String ticket(String[] names, int[] pricesCents) {
    StringBuilder sb = new StringBuilder("=== FoodFast ===\n");
    for (int i = 0; i < names.length; i++) {
        sb.append(names[i]).append(" : ")
          .append(formatEuros(pricesCents[i])).append('\n');
    }
    return sb.toString();
}
// ticket({"Margherita", "Tiramisu"}, {1250, 650})
```

📖 Support §4.2 — StringBuilder

🔒 Ne jamais faire confiance à une saisie

Une quantité saisie arrive sous forme de **texte** : `"3"`, `" 3"`, `"abc"`, `"-2"`, `"99999999999"`, `"٣"`…

1. **Présence** : ni `null`, ni vide.
2. **Format** : uniquement des chiffres `0` à `9`, longueur maximale.
3. **Bornes** : une quantité entre 1 et 20.
4. Seulement **ensuite**, convertir et calculer.

> On décrit ce qui est **autorisé** (liste blanche), pas ce qui est interdit.

📖 Support §4.3 — Valider les entrées

🔒 Valider une quantité saisie

```
static final int MAX_QUANTITY = 20;

static boolean isValidQuantity(String input) {
    if (input == null || input.isBlank() || input.length() > 2) {
        return false;
    }
    for (char c : input.toCharArray()) {
        if (c < '0' || c > '9') {
            return false;
        }
    }
    int quantity = Integer.parseInt(input);
    return quantity >= 1 && quantity <= MAX_QUANTITY;
}
```

📖 Support §4.3 — Valider les entrées

Convertir un texte en nombre, sans planter

```
static String addQuantity(long current, String text) {
    try {
        long delta = Long.parseLong(text);
        return "Total : " + Math.addExact(current, delta);
    } catch (NumberFormatException e) {
        return "Erreur : ce n'est pas un nombre entier";
    } catch (ArithmeticException e) {
        return "Erreur : dépassement de capacité";
    }
}
// addQuantity(10, "5")              → Total : 15
// addQuantity(10, "abc")            → Erreur : ce n'est pas un nombre…
// addQuantity(Long.MAX_VALUE, "1")  → Erreur : dépassement de capacité
```

📖 Support §4.6 — Long.parseLong et ses exceptions

🛠️ À vous — Partie 4 (25 à 35 min)

1. `static String normalizePromoCode(String raw)` :
   `"  promo10 "` → `"PROMO10"`, et `null` → `""`.
2. `static int parseQuantity(String text)` : la quantité (1 à 20), ou `-1`.
   `Integer.parseInt` dans un `try` / `catch`. Essayez `"3"`, `"abc"`, `"21"`, `null`…
   🔒 et `"٣"` (copiez-le) : est-il accepté ? Est-ce normal ?
3. Un ticket avec `StringBuilder` :

```
2 x Margherita = 25,00 €
1 x Tiramisu = 6,50 €
TOTAL = 31,50 €
```

4. 🔵 Affichez `"Crème brûlée".length()`, puis `"Crème brûlée".getBytes().length`.
   Pourquoi ?

5. Maven et JUnit

Maven : construire un projet, pas un fichier

- **Construit** : `mvn test` compile puis lance les tests ; `mvn package` crée le `.jar`.
- **Télécharge les dépendances** (JUnit…) depuis Maven Central.
- **Impose une structure** : tout projet Maven se ressemble.

```
foodfast/
├── pom.xml                                  ← description du projet
└── src/
    ├── main/java/fr/foodfast/
    │   └── OrderCalculator.java             ← code
    └── test/java/fr/foodfast/
        └── OrderCalculatorTest.java         ← tests
```

📖 Support §5.1, §5.2 et §5.4 — Maven

`pom.xml` : Java 25 et JUnit

```xml
<project>
    <groupId>fr.foodfast</groupId>          <!-- les coordonnées -->
    <artifactId>foodfast</artifactId>       <!-- du projet       -->
    <version>1.0-SNAPSHOT</version>
    <properties>
        <maven.compiler.release>25</maven.compiler.release>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>6.1.3</version>
            <scope>test</scope>             <!-- pour les tests -->
        </dependency>
    </dependencies>
</project>
```

- Dans le TP, le `pom.xml` est **fourni** : à savoir lire, pas à écrire.

📖 Support §5.3 — Le pom.xml complet

Un premier test JUnit

```java
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class OrderCalculatorTest {
    @Test
    void total_de_deux_pizzas_et_une_boisson() {
        int[] prices = {1250, 350};                          // Given
        int[] quantities = {2, 1};

        int total = OrderCalculator.totalCents(prices, quantities); // When

        assertEquals(2850, total);                           // Then
    }
}
```

📖 Support §5.5 — Premier test JUnit

Les assertions, et lire le résultat

```
assertEquals(2850, total);                          // attendu, obtenu
assertTrue(InputValidator.isValidQuantity("3"));
assertFalse(InputValidator.isValidQuantity("21"));
assertThrows(IllegalArgumentException.class,
        () -> OrderCalculator.applyDiscount(2850, 150));
```

```
Tests run: 12, Failures: 1, Errors: 0, Skipped: 2
BUILD FAILURE
```

- **Failures** : une assertion fausse. **Errors** : une exception imprévue.
- **Skipped** : un test désactivé exprès par `@Disabled`.
- `() -> …` : « le code à exécuter », pour vérifier qu'il lève l'exception. Détails en séance 3.

📖 Support §5.5 et §5.7 — Assertions et tests d'exception

Écrire de bons tests

- **Given / When / Then** : préparer, agir, vérifier.
- **Un comportement par test**, et un nom qui le décrit.
- Tester le **cas nominal**, les **cas limites** (`0`, `1`, `20`, `21`, tableau vide)…
- … et les **entrées invalides** (`"abc"`, `null`, `"-2"`).

```
@Test
void une_quantite_de_21_est_refusee() {
    assertFalse(InputValidator.isValidQuantity("21"));
}
```

📖 Support §5.6 — Écrire de bons tests

🛠️ À vous — Partie 5 (30 à 40 min)

1. Dans le projet Maven d'IntelliJ, `src/main/java/BasketStats.java` :

```
public class BasketStats {
  static int countAbove(int[] prices,
                        int threshold) {
    int count = 0;
    for (int p : prices) {
      if (p > threshold) {
        count++;
      }
    }
    return count;
  }
}
```

2. Curseur sur `BasketStats`, **Alt+Entrée** → *Create Test* (JUnit 5, bouton *Fix* si proposé).

```
@Test
void prix_egal_au_seuil_non_compte() {
  int[] prices = {1250, 1000, 450};
  assertEquals(1,
      BasketStats.countAbove(prices, 1000));
}
```

3. ▶▶ dans IntelliJ, puis `mvn test`.

4. Cassez : `>` devient `>=`. Lisez `expected` / `but was`.

5. Deux tests à vous.

Ce qu'il faut retenir

- `.java` → **`javac`** → bytecode → **JVM**. Java 25 : `void main()` et `IO.println`.
- Dans un projet : une **classe nommée**, testable.
- Montants en **centimes** (`int`) aujourd'hui, `BigDecimal` en séance 2.
- `String` : **`equals`**, jamais `==`. `StringBuilder` dans les boucles.
- 🔒 Débordement : `Math.addExact` / `multiplyExact`. Entrées : présence, format, bornes.
- `mvn test` + JUnit : **Given / When / Then**, cas limites, entrées invalides.

Pour aller plus loin

Dans le **support détaillé** de la séance 1 :

- 🔵 Text blocks, `import module`, surcharge, portée des variables
- 🔵 Tests paramétrés (`@ParameterizedTest`)
- 🟣 **Défis** : algorithme de Luhn, robustesse d'un mot de passe, chiffre de César, FizzBuzz piloté par les tests, `switch` à motifs sur un `record`, code promo avec `SecureRandom`

> Séance 2 : de la donnée à l'objet. Classes, `record`, `enum`, `BigDecimal`, collections.

Le TP : ShortLink 🔗

Un **raccourcisseur de liens**, comme bit.ly, construit sur les 4 séances :
console et règles de validation aujourd'hui, objets en S2, fichiers et streams en S3, serveur web, concurrence et SQL en S4.

- Chaque comportement attendu est décrit par un **test fourni** : vous les activez un par un.
- 🔵 🟣 pour aller plus loin, 🔒 pour la sécurité.

🟣 **Parcours autonome, pour les expérimentés** : **Mini-Redis**, un serveur compatible Redis, sans guidage pas à pas. Vous avancez seuls avec ses outils ; même évaluation.

Pour le TP

Aujourd'hui, vous réutiliserez :

- `switch` en expression sur une `String`, méthodes `static`
- `List` et `Map`, `String` et `StringBuilder`
- valider une entrée par liste blanche 🔒
- `Math.addExact` contre le débordement 🔒
- JUnit : un test par comportement, sans oublier les entrées invalides

> Lisez les tests fournis : ils sont la spécification.

🛠️ TP — Prise en main (25 min)

1. **Récupérer** : github.com/mehdi-elke/java-lecture → *Code* → *Download ZIP*.
   Copiez `tp/shortlink/starter` où vous voulez, renommez-le `shortlink`.
2. **L'ouvrir dans IntelliJ**, puis `mvn test` : `BUILD SUCCESS`, avec 40 tests *skipped* (normal).
3. **Suivre le §0 de l'énoncé** : `tp/shortlink/enonce/seance1.md`.

🟣 **Parcours autonome Mini-Redis** : `tp/miniredis/starter`, énoncé `tp/miniredis/enonce/seance1.md`.



Pause — 15 minutes

Reprise : **TP, §1 de l'énoncé**

QCM diagnostique — 10 minutes

- **Non noté.** Il me dit ce qu'on reprend au début de la séance 2.
- Sur une feuille : votre prénom (facultatif), puis une lettre par question.
- `1–B  2–D  3–A …`
- Seul, sans IA, sans le support : c'est **votre** niveau qui m'intéresse.

Le rendu de la séance 1

À rendre : la veille de la séance 2, à 23h59, sur un dépôt GitHub **privé**.

- Terminez les exercices 🟢 de l'énoncé.
- `mvn test` qui passe, et des tests JUnit à vous.
- Un commit par exercice, poussé sur GitHub.
- Invitez l'enseignant : **`mehdi-elke`**.

En séance 2 : **3 minutes avec moi** pour expliquer et modifier votre code.

📄 Tout est détaillé dans `tp/<sujet>/enonce/rendu-seance1.md`.